

Lab 02: Multiple and Logistic regression

Author: **N.J. de Winter** (n.j.de.winter@vu.nl)

Assitant Professor Vrije Universiteit Amsterdam

Statistics and Data Analysis Course

Learning goals:

- Practice your skills interpreting and modifying Python code
- Understand and apply tools to test which *independent variables* in a larger dataset are relevant to predict the *dependent variable* (multiple regression)
- Learn how to apply *regression* analysis to determine the *relationship* between multiple variables:
 - Multiple linear regression (straight line relationships between multiple variables)
 - Logistic regression (Logistic, or *sigmoid*-shaped relationships between multiple variables)

Introduction

In this lab, we will get familiar with multiple regression and logistic regression using Python. Make sure you have completed Lab 01 first before starting on this one, because we will assume all the concepts in the first lab are familiar to you now. As a consequence, in this lab, there will be less "hand holding" and you will need to figure out more Python coding yourself. We will also assume from now on that the Jupyter format is familiar to you, although you are also still free to run the code in your own Spyder environment. If you ever get lost trying to get something done in Python, you can always go back to Lab 01 to check for examples about how to code things.

We will start again by loading the Python *packages* we need in this lab:

```
In [ ]: # Make sure our figures show up in Jupyter
%matplotlib inline
import numpy as np # The 'numpy' package contains some handy functions
import pandas as pd # The 'pandas' package helps us to import and manage data
import scipy.stats as stats # The 'scipy' package contains statistical formulas we need
import statsmodels.api as sm # The 'statsmodels' package contains the functions we need to run regression models
from matplotlib import pyplot as plt # The 'matplotlib' package contains tools need to plot our results
from mpl_toolkits.mplot3d import Axes3D # This is a cool package for making 3D-plots!
```

Part 1: Multiple regression

With that out of the way, we can load the dataset we need to do our multiple regression. We will start with the dataset `Lab02a.csv`.

Exercise 1: Load the dataset here using the code box below. If you don't remember how to do this, look back to Lab01 and use the code you used there to load and inspect the data.

Warning: If you are working on this Lab (Lab02) in a different folder or environment on your computer than on Lab01 (which is a good idea!), make sure the data file is in the right place.

```
In [ ]: # Load the data for this assignment into Python and in the Jupyter environment.

# Inspect the first rows of the dataset to familiarize yourself again with this data
```

This data file lists measurements of organic matter content in soils. It contains the following variables:

- `x` (the x coordinate of the location)
- `y` (the y coordinate of the location)
- `height` (the height of the soil at these locations)
- `perc_org` (the percentage organic matter in the top 10 cm of the soil at these locations)

Soil organic matter content varies depending on location. We want to verify if there is systematic variability. For example, soil organic matter content may change in a certain direction because of elevation changes. In this case, we should be able to detect this difference along the x or y coordinates. If soil organic matter content increases in the x direction, there should be a positive correlation between soil organic matter content and x coordinates. If soil organic matter content increases along the diagonal between x

and y coordinates, there should be positive correlations between soil organic matter content and both x and y. We could express this using a regression equation with both x and y as independent variables. First, we will check the relationship between soil organic matter, and the x and y coordinates.

Question 1: How can we check whether there is a correlation between soil organic matter and x or y coordinates?

Answer 1:

In Lab01, we have assessed the correlations between variables one by one. This is feasible for relatively small datasets like this one, but quickly becomes tedious for larger datasets. Below you find a more efficient code to check correlations between all variables in a dataset. For it to work we first need to isolate the variables we are interested in:

```
In [ ]: df3_2 = df3.iloc[:, 0:3] # Isolate the first three columns of df3
```

Exercise 2: Inspect the first rows of the newly created dataframe to make sure the function did what you wanted it to do:

```
In [ ]: # inspect the new dataframe `df2_3`
```

Now we can use the function `corr()` to calculate correlation coefficients between the variables.

NOTE: This function `.corr` only works on `pandas` dataframes (loaded using `pd.read_csv()`), for more general purposes you can use the function `np.corrcoef()` from the `numpy` package.

```
In [ ]: A = df3_2.corr() # Create correlation matrix
print(A) # Print the result
```

Notice that this is in fact the same function we used in Lab01, but without specifying the variables.

Question 2: Does organic matter content correlate most strongly with the x or y coordinate? Why are there ones at the diagonal of the output matrix?

Answer 2:

Exercise 3: Plot scatter plots between variables `x`, `y` and `perc_organic`. (Hint: have a look at the code in Lab01 if you do not remember how to do this).

```
In [ ]: # Plot perc_org vs x coordinate
```

```
In [ ]: # Plot perc_org vs y coordinate
```

```
In [ ]: # Plot x vs y coordinate
```

Preparing data for multiple regression

We will now explore how well the combination of the two coordinates can predict the organic matter content in the soil. We will use a *multiple linear regression* to achieve this. Running a multiple linear regression in Python is actually not that different from running a simple linear regression, but we need to prepare our data a little bit first:

```
In [ ]: # First we isolate the dependent variable (percentage organic matter)
Y = df3_2[['perc_org']]
```

```
In [ ]: # Then we group the independent variables (coordinates)
X = df3_2[['x', 'y']]
```

```
In [ ]: # Now, as with the polynomial regression, we need to add a constant (vector of ones) to tell Python that we want to include an intercept value in our regression
# The statsmodels package has a neat function to do this:
X = sm.add_constant(X)
```

Exercise 4: Use the functions you have used before to inspect the newly created datasets `X` and `Y`:

```
In [ ]: # Inspect dataframe `X`
```

```
In [ ]: # Inspect dataframe `Y`
```

For our multiple regression, we will use a slightly different function from the `statsmodels` package than the one we used in Lab01. The syntax is almost the same as `smf.ols`:

```
In [ ]: multreg1 = sm.OLS(Y, X).fit()
```

Exercise 5: Use the code box below to print a summary of the multiple regression. (Hint: Use the same code you used in Lab01 to print summaries of simple linear regressions)

```
In [ ]: # Print a summary of the multiple linear regression
```

Question 3: Looking at the multiple regression overall, does the combination of independent variables accurately predict the value of the independent variable? Which regression metrics do you use to answer this question?

Answer 3:

Question 4: Considering the contributions of the independent variables (x and y coordinates) separately, which variables significantly influence the dependent variable (organic matter content)? Which metrics do you use to answer this question?

Answer 4:

We will now use some of Python's visualization capabilities to better understand the meaning of the regression coefficients. Our regression estimates the soil organic matter content P based on the equation $P = a + bx + cy$. This equation defines a surface in a 3-dimensional space with the axes x, y and P. The surface has to be calculated on equally spaced grid in x and y and is saved as zdata. We use the `mpl_toolkits.mplot3d` package to achieve this. The zdata needs input coefficients from the multilinear regression for the constant, and the x and y variables. We can find these coefficients from the regression results using the `params` function.

Exercise 6: To display the 3D surface, follow the steps below. Make sure you inspect the newly created dataframes so you understand every step.

```
In [ ]: # Prepare the equally spaced grid of x and y coordinates
xdata = range(int(np.floor(min(df3.x))), int(np.ceil(max(df3.x))), 1) # Create range of x values at equal intervals
ydata = range(int(np.floor(min(df3.y))), int(np.ceil(max(df3.y))), 1) # Create range of y values at equal intervals
grid = np.array([(x, y) for x in xdata for y in ydata]) # Combine the two ranges into a grid
```

```
In [ ]: # Inspect new x-y grid
print(grid)
```

```
In [ ]: # Calculate the z-data for each combination of x and y
zdata = multreg1.params[0] + multreg1.params[1] * grid[:, 0] + multreg1.params[2] * grid[:, 1]
```

```
In [ ]: # Inspect the new z data
print(zdata)
```

```
In [ ]: # Plot the 3D-plot
plt.figure(2) # initiate the plot
ax = plt.axes(projection = '3d') # Tell Python to create a 3D projection
ax.plot_trisurf(grid[:, 0], grid[:, 1], zdata, cmap = 'summer') # Create a 3D surface plot with values colored according to the "summer" colorscale
ax.scatter(df3.x, df3.y, df3.perc_org, c=df3.perc_org, cmap='summer') # Add the datapoints from which the surface is calculated in the same color scheme
ax.set_xlabel('x-coordinates') # Label the x axis
ax.set_ylabel('y-coordinates') # Label the y axis
ax.set_zlabel('Organic Matter [%]') # Label the z axis
```

Maybe the soil organic matter content is not really determined by the x or y coordinates, but by another variable. Let's add height as a variable.

Exercise 7: Redo **Exercise 2** and **Exercise 3** after adding the height variable. Start by creating a scatterplot of height vs. organic matter percentage.

WARNING: Be careful naming your objects to prevent overwriting something you may need later or confusing data!

```
In [ ]: # Plot perc_org vs y coordinate
```

```
In [ ]: # Prepare our data
```

```
# First we isolate the dependent variable (percentage organic matter)
```

```
# Then we group the independent variables (coordinates)
```

```
# Now we add a constant (vector of ones) to tell Python that we want to include an intercept value in our regression
```

```
# Run the regression
```

```
# Print a summary of the new multiple linear regression
```

Question 5: What do you observe about the regression overall? Do the x and y coordinates still provide a statistically significant contribution at $p < 0.05$?

Answer 5:

In the end, elevation is the main determinant of soil organic matter content in our case study. Lower elevations have higher soil organic matter content. This may be because lower elevation locations are wetter and support more vegetation, and thus production of organic matter. It also takes longer to decompose organic matter in wet soils compared to dry soils. Another reason could be that erosion processes denude organic matter from higher elevation locations and deposit them at lower elevation locations.

The lesson here is that (multiple) regressions can only give you insight into the relationships between variables you add to the regression. If a variable (like `height` in this case) is missing, you may draw the wrong conclusions. Inspecting scatter plots and correlation matrices of all variables before (blindly) trying to do a regression could have warned you for this.

Part 2: Logistic regression

We will practice with *Logistic regression* using the dataset `Lab02b.csv`. Use the code box below to load it using Python code you are now familiar with

```
In [ ]: # Load the data for this assignment into Python and in the Jupyter environment.
```

We also need some additional functions and packages. The code below should load them for you.

```
In [ ]: import statsmodels.api as sm
        from sklearn.linear_model import LogisticRegression
        import seaborn as sns
```

Once you have imported the data, use the code box below to inspect it.

```
In [ ]: # Inspect the new dataframe
```

Question 6: Are there any values in this dataset which may cause you problems when doing data analysis?

Answer 6:

It is likely that you did not find any problems with the dataset through your inspection. However, the dataset does in fact contain missing values (reported by Python as `NA`), but the dataset is large and they are somewhat hidden. You can use the combination of functions below to find them:

```
In [ ]: df4.isna().sum()
```

Exercise 8: Now that you know this, you may remove missing values using the `dropna()` command. You can do this in a similar way as when you inspect your dataset using `.head()` or when you detected the `NA` values with `.isna` and `sum()` above. Try it, and then verify that the `NA`'s are gone using the same function you used above.

```
In [ ]: # Remove the missing values from the dataset
        # Check whether the new dataset has no NA's
```

The data file contains multiple variables of performance metrics from current and former basketball players in the NBA. For clarification, the variables abbreviations are explained in Table 1. Take a good look at these variables.

Table 1 Abbreviation and explanation of different variables of NBA players.

Abbreviations	Explanation
GP	Games played
MIN	Minutes played per game
PTS	Points per game
FG_PC	Field goal % (shooting accuracy) per game
THREEP_Made	3 pointer made per game
THREEP_PC	3 pointer % (3 pointer shooting accuracy) per game
FTM	Free throw made per game

Abbreviations	Explanation
FT_PC	Free throw % (free throw shooting accuracy) per game
REB	Rebounds per game
AST	Assists per game
TARGET_5Yrs	Players lasted more than 5 years in the league or not
STL	Steals per game
BLK	Blocks per game
TOV	Turnovers per game (amount of lost balls)

A number of variables are expressed as percentages, while others are expressed as a number of actions per game.

Question 7: Do you think it is fair to compare performance per game knowing that the number of minutes played per game varies among the players?

Answer 7:

To normalize for the number of minutes played per game, we will divide the `PTS`, `THREEP_Made`, `FTM`, `REB`, `AST`, `STL`, `BLK` and `TOV` by the `MIN` variable to derive the number of actions per minute. For the `PTS` variable, you can for example do this by using the following command and check the result using the `.head()` function, as you are used to:

```
In [ ]: df4_2['PTS_MIN'] = df4_2['PTS'] / df4_2['MIN']
df4_2.head()
```

NOTE: The code above might trigger a "SettingWithCopyWarning", which is a warning by Python that you are making changes to your dataframe by modifying copies of elements of it (in this case the columns). You can ignore this warning for now, but if you are interested to find out why Python warns you for this you can look it up online. If you want to ignore this warning throughout the Python session, you can use the following code:

```
pd.options.mode.chained_assignment = None # default='warn'
```

Exercise 9: Normalize the other variables which depend on the number of minutes played in the same way using the code block below and inspect the result.

```
In [ ]: # Ignore the annoying warning
pd.options.mode.chained_assignment = None # default='warn'

# Normalize values

# Inspect results
```

We will continue our analysis with the following variables: `GP`, `MIN`, `PTS_MIN`, `FG_PC`, `THREEP_Made_MIN`, `THREE_PC`, `FTM_MIN`, `FT_PC`, `REB_MIN`, `AST_MIN`, `TARGET_5Yrs`, `STL_MIN`, `BLK_MIN`, `TOV_MIN`. The aim of the exercise is to tell which performance metrics influence whether a player lasts more than five years in the NBA league.

Logistic regression analysis is used to predict the outcome of a dependent categorical variable. In other words, the regression analysis assesses the chances that the outcome of the dependent variable is either 1 (yes, success, etc.) or 0 (no, failure, etc.). Hence, the dependent variable is a binary variable.

Question 8: Based on your knowledge of logistic regression and the data file (Table 1). Which variable is the binary dependent variable we will predict?

Answer 8:

The dataset is extensive and complex, but could easily be expanded with more variable variables (e.g. the position on the basketball court, age, etc.). Can you think of other variables? With a logistic regression, you can assess which variables have influence on a player's success to last longer than five years in the NBA league.

First, we will explore which performance metrics may influence a player's success to last longer than 5 years in the NBA league. You can group players based on whether or not they played longer than 5 years in the NBA. After this, you can plot the values of the different performance metrics for those players that played longer than five years in the NBA with those that did not. You can use boxplots to visualize the results. For the `GP` variable, this for example results in the following command:

```
In [ ]: sns.boxplot(x = df4_2.TARGET_5Yrs, y = df4_2.GP, data = df4_2)
```

Exercise 10: Do the same for all other performance metrics.

```
In [ ]: # Plot boxplots of all other variables
```

Question 9: What do you observe? Which performance metrics seem to influence a player's success to last longer than five years in the NBA league? Do the variables with positive and negative influences seem logical? Does any of the findings seem counterintuitive?

Answer 9:

The function `sns.countplot` is a good way to investigate which number of players made it for more than five years in the NBA league:

```
In [ ]: sns.countplot(x = df4_2.TARGET_5Yrs, data = df4_2)
```

Now, we will perform a logistic regression analysis with the `statsmodels` function `sm.Logit` which you loaded earlier.

Exercise 11: We first need to define all the predictors (independent variables) and the dependent variables. You can do this in the same way as you did in the multiple regression exercises above (if you forgot, check the code above **Exercise 2**):

```
In [ ]: # Define independent variables
# Define dependent variables
```

After this, we can perform the logistic regression and print its results:

```
In [ ]: logit_model = sm.Logit(y,X).fit()
print(logit_model.summary2())
```

Question 10: Which performance metrics have a significant influence at a player's success to last longer than five years in the NBA league at $p < 0.05$?

Answer 10:

Before performing another logistic regression model with less variables, it is good to understand the information in the regression summary. In the summary, the coefficient values are values that describe how each predictor influences the outcome of a player lasting longer than five years in the NBA league. For example, the variable games played (`GP`) is significant at $p < 0.05$. This tells you that for every unit increase in `GP`, the log odds of lasting longer than years (versus lasting less than 5 years) increases by 0.035.

Another variable, turnovers per game divided by minutes played (`TOV_MIN`), also significantly contributes in the model at $p < 0.05$. For every unit increase in the `TOV_MIN`, the log odds of a player lasting longer than five years in the NBA league *decreases* with 10.391 (it has a negative coefficient). Thus, numeric increases in some performance metrics positively influence the odds of lasting longer than 5 years in the NBA. For other performance metrics, increases in numeric values negatively influence the chance of lasting longer than five years in the NBA league.

Question 11: Which significant predictor variables, at $p < 0.05$, show the largest positive and negative influence on a player staying in the NBA for more than five years? Which predictors variables do not have a significant influence at $p < 0.05$ and could thus be taken out of the model? Are any of the findings counterintuitive?

Answer 11:

Exercise 12: Execute another logistic regression with only the performance metrics that were significant at $p < 0.05$ in the previous regression.

```
In [ ]: # First define independent and dependent variables

# Then run the new regression
```

Question 12: Does this lead to big changes in the coefficients and significance of the remaining performance metrics in comparison with the previous regression? Based on our analysis, how would you describe the profile a player of a player with a good chance of lasting longer than 5 years in the NBA?

Answer 12: